

File Systems

A Semester Project-II

This article, which is part of the series on Linux device drivers, elaborates on the concept of a file system by simulating one in user space.



Part—19

In the previous article, Shweta readied the partition on the .sfs file by formatting it with the *format_sfs* application. To completely understand a file system, the next step in its simulation is to browse and play around with the file system created. Here is Shweta's first-cut browser application, for precisely that purpose. Let's have a closer look. *sfs_ds.h* is the header file already created by Shweta.

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <time.h>

#include "sfs_ds.h"

sfs_super_block_t sb;

void sfs_list(int sfs_handle)
{
    int i;
    sfs_file_entry_t fe;

    lseek(sfs_handle, sb.entry_table_block_start * sb.block_size,
    SEEK_SET);
    for (i = 0; i < sb.entry_count; i++)
    {
        read(sfs_handle, &fe, sizeof(sfs_file_entry_t));
        if (!fe.name[0]) continue;
        printf("%-15s %10d bytes %c%c%c %s",
            fe.name, fe.size,
            fe.perms & 04 ? 'r' : '-',
            fe.perms & 02 ? 'w' : '-',
            fe.perms & 01 ? 'x' : '-',
            ctime((time_t *)&fe.timestamp)
        );
    }
}
```

```
}
void sfs_create(int sfs_handle, char *fn)
{
    int i;
    sfs_file_entry_t fe;

    lseek(sfs_handle, sb.entry_table_block_start * sb.block_size,
    SEEK_SET);
    for (i = 0; i < sb.entry_count; i++)
    {
        read(sfs_handle, &fe, sizeof(sfs_file_entry_t));
        if (!fe.name[0]) break;
        if (strcmp(fe.name, fn) == 0)
        {
            printf("File %s already exists\n", fn);
            return;
        }
    }
    if (i == sb.entry_count)
    {
        printf("No entries left\n", fn);
        return;
    }

    lseek(sfs_handle, -sb.entry_size, SEEK_CUR);

    strncpy(fe.name, fn, 15);
    fe.name[15] = 0;
    fe.size = 0;
    fe.timestamp = time(NULL);
    fe.perms = 07;
    for (i = 0; i < SIMULA_FS_DATA_BLOCK_CNT; i++)
    {
        fe.blocks[i] = 0;
    }
    write(sfs_handle, &fe, sizeof(sfs_file_entry_t));
}

void sfs_remove(int sfs_handle, char *fn)
{
}
```